

School of Computer Science and Artificial Intelligence

Lab Assignment # 16

Name of Student : Chathraboina Sagar
Enrollment No. : 2303A51522
Batch No. : 22

Task -: (Design Database Schema for Student Management System)

Instructions:

- Using AI tools, generate CREATE TABLE statements for a Student Management Database.
 - Tables: Students, Courses, Enrollments.
 - Include primary keys, foreign keys, and suitable data types.
- Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Sample Code:-

```
CREATE TABLE Students (student_id INT PRIMARY KEY, name VARCHAR(100), email VARCHAR(100));  
CREATE TABLE Courses (course_id INT PRIMARY KEY, course_name VARCHAR(100), credits INT);  
CREATE TABLE Enrollments (enrollment_id INT PRIMARY KEY, student_id INT, course_id INT, FOREIGN KEY (student_id) REFERENCES Students(student_id), FOREIGN KEY (course_id) REFERENCES Courses(course_id));
```

Expected Output:-

Tables Students, Courses, and Enrollments are created successfully.
Foreign key constraints are properly enforced.

Prompt:-

Create a database schema design for a **Student Management System**. Include SQL CREATE TABLE statements for students, courses, and enrollments. Ensure that the schema supports a **many-to-many relationship** between students and courses. The design should include appropriate **primary keys and foreign keys**, and use suitable **data types** for each field. The schema must follow **normalization rules (1NF, 2NF, 3NF)** to eliminate redundancy and maintain data integrity.

Code:

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    course_code VARCHAR(10) UNIQUE NOT NULL
);
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id) -- To prevent duplicate enrollments
);
```

```
insert into Students (first_name, last_name, email) values ('John', 'Doe', 'john.doe@example.com');
insert into Courses (course_name, course_code) values ('Mathematics', 'MATH101');
insert into Enrollments (student_id, course_id, enrollment_date) values (1, 1, '2024-01-15');
show tables;
select * from Students;
select * from Courses;
select * from Enrollments;
```

Output:-

SQL ▾< 1 / 1 > 1 - 1 of 1🗑️↶↷🔍

student_id	first_name	last_name	email
NULL	John	Doe	john.doe@example.com

SQL ▾< 1 / 1 > 1 - 1 of 1🗑️↶↷🔍

course_id	course_name	course_code
NULL	Mathematics	MATH101

SQL ▾< 1 / 1 > 1 - 1 of 1🗑️↶↷🔍

enrollment_id	student_id	course_id	enrollment_date
NULL	1	1	2024-01-15

Justification:-

The schema is designed to support a **Student Management System** with three tables: Students, Courses, and Enrollments. The Students table includes a primary key (student_id) and fields for first name, last name, and email, ensuring that each student has a unique identifier and contact information. The Courses table includes a primary key (course_id) and fields for course name and course code, allowing for unique identification of each course. The Enrollments table establishes a many-to-many relationship between students and courses through foreign keys (student_id and course_id) and includes an enrollment date. The UNIQUE constraint on the combination of student_id and course_id prevents duplicate enrollments, ensuring data integrity. This design adheres to normalization rules (1NF, 2NF, 3NF) by eliminating redundancy and ensuring that each piece of data is stored in the appropriate table.

Task -2-: (Insert Sample Records)

Instructions: Use AI tools to generate INSERT statements to populate the tables.

- Insert at least 3 students, 4 courses, and 5 enrollment records.

Verify data using SELECT queries.

Sample Code:-

```
INSERT INTO Students VALUES(1, 'Arjun', 'arjun@gmail.com'), (2, 'Meera', 'meera@gmail.com'), (3, 'Rahul', 'rahul@gmail.com');  
INSERT INTO Courses VALUES (101, 'Database Systems', 4), (102, 'Data Structures', 3), (103, 'Operating Systems', 4), (104, 'AI Fundamentals', 3);  
INSERT INTO Enrollments VALUES (1, 1, 101), (2, 1, 102), (3, 2, 103), (4, 3, 101), (5, 3, 104);
```

Expected Output:-

- Sample records are inserted successfully.
- FROM Students; displays all student records.
- SELECT * FROM Enrollments; displays all enrollments.

Prompt:-

Insert sample data into the Students, Courses, and Enrollments tables of the Student Management System. Add at least 3 students, 4 courses, and 5 enrollment records, then display the tables to verify the data insertion.

Code:-

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL  
);  
CREATE TABLE courses (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(100) NOT NULL,  
    course_code VARCHAR(10) UNIQUE NOT NULL  
);  
CREATE TABLE enrollments (  
    enrollment_id INT PRIMARY KEY,  
    student_id INT,  
    course_id INT,  
    enrollment_date DATE,  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),  
    UNIQUE(student_id, course_id) -- To prevent duplicate enrollments  
);  
  
insert into Students (first_name, last_name, email) values ('John', 'Doe', 'john.doe@gmail.com');  
insert into Students (first_name, last_name, email) values ('Jane', 'Smith', 'jane.smith@gmail.com');  
insert into Students (first_name, last_name, email) values ('Bob', 'Johnson', 'bob.johnson@gmail.com');  
insert into Courses (course_name, course_code) values ('Mathematics', 'MATH101');  
insert into Courses (course_name, course_code) values ('Physics', 'PHYS101');  
insert into Courses (course_name, course_code) values ('Chemistry', 'CHEM101');  
insert into Courses (course_name, course_code) values ('Biology', 'BIO101');  
insert into Enrollments (student_id, course_id, enrollment_date) values (1, 1, '2024-01-15');  
insert into Enrollments (student_id, course_id, enrollment_date) values (1, 2, '2024-01-16');  
insert into Enrollments (student_id, course_id, enrollment_date) values (2, 1, '2024-01-17');  
insert into Enrollments (student_id, course_id, enrollment_date) values (2, 3, '2024-01-18');  
insert into Enrollments (student_id, course_id, enrollment_date) values (3, 4, '2024-01-19');  
  
select * from students;  
select * from courses;  
select * from enrollments;
```

Output:-

SQL ▾

< 1 / 1 > 1 - 3 of 3

   

student_id	first_name	last_name	email
NULL	John	Doe	john.doe@gmail.com
NULL	Jane	Smith	jane.smith@gmail.com
NULL	Bob	Johnson	bob.johnson@gmail.com

SQL ▾

< 1 / 1 > 1 - 4 of 4

   

course_id	course_name	course_code
NULL	Mathematics	MATH101
NULL	Physics	PHYS101
NULL	Chemistry	CHEM101
NULL	Biology	BIO101

SQL ▾

< 1 / 1 > 1 - 5 of 5

   

enrollment_id	student_id	course_id	enrollment_date
NULL	1	1	2024-01-15
NULL	1	2	2024-01-16
NULL	2	1	2024-01-17
NULL	2	3	2024-01-18
NULL	3	4	2024-01-19

Justification:-

The above SQL code creates three tables: Students, Courses, and Enrollments, with appropriate primary keys, foreign keys, and suitable data types. It then inserts sample data into each table to demonstrate the many-to-many relationship between students and courses. Finally, it retrieves and displays the contents of each table to verify that the data has been inserted correctly.

Task -3:- (Perform CRUD Operations)

Instructions:-

Use AI to generate SQL queries for basic CRUD operations:

- Retrieve all courses
- Update a student's email
- Delete an enrollment record
- Validate the changes using SELECT statements.

Sample Code:-

```
SELECT * FROM Courses;
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
DELETE FROM Enrollments
WHERE enrollment_id = 5;
```

Expected Output:

- Updated email is reflected in Students table.
- Deleted enrollment record is removed successfully.

- Remaining data remains unchanged.

Prompt:-

Write SQL queries to perform basic CRUD operations such as retrieving all courses, updating a student's email, and deleting an enrollment record. Also, validate the changes using appropriate SELECT statements.

Code:-

```
-- Task-3
-- SQL Queries for basic CRUD operations li
select * from courses;
UPDATE students
SET email = 'john.doe.updated@gmail.com'
WHERE student_id = 1;

DELETE FROM enrollments
WHERE enrollment_id = 1;

select * from students;
select * from enrollments;
```

Output:-

Output

SQL ▾

< 1 / 1 > 1 - 4 of 4



course_id	course_name	course_code
NULL	Mathematics	MATH101
NULL	Physics	PHYS101
NULL	Chemistry	CHEM101
NULL	Biology	BIO101

SQL ▾

< 1 / 1 > 1 - 0 of 0



```
UPDATE students
SET email = 'john.doe.updated@gmail.com'
WHERE student_id = 1;
```

SQL ▾

< 1 / 1 > 1 - 0 of 0



```
DELETE FROM enrollments
WHERE enrollment_id = 1;
```

SQL ▾

< 1 / 1 > 1 - 3 of 3



student_id	first_name	last_name	email
NULL	John	Doe	john.doe@gmail.com
NULL	Jane	Smith	jane.smith@gmail.com
NULL	Bob	Johnson	bob.johnson@gmail.com

SQL ▾

< 1 / 1 > 1 - 5 of 5



enrollment_id	student_id	course_id	enrollment_date
NULL	1	1	2024-01-15
NULL	1	2	2024-01-16
NULL	2	1	2024-01-17
NULL	2	3	2024-01-18
NULL	3	4	2024-01-19

Justification:-

The above SQL queries demonstrate basic CRUD operations on the Student Management System database. The first query retrieves all courses, allowing us to view the available courses in the system. The second query updates a student's email address, ensuring that accurate contact information is maintained. The third query deletes an enrollment record, which helps manage course enrollments and ensures that the database reflects current student-course relationships. Finally, the SELECT statements validate the changes by displaying the updated student information and the remaining enrollment records.

Task -4:- (Execute Aggregate Queries)

Instructions: Use AI tools to write aggregate queries:

- Count number of students enrolled per course
 - Display courses with more than one student enrolled
- Analyze the correctness of results.

Sample Code:

```
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Expected Output:

- Correct count of students per course is displayed.
- Courses with multiple enrollments are listed accurately.

Prompt:-

Write SQL aggregate queries to count the number of students enrolled in each course and display courses with more than one student enrolled. Also, analyze the correctness of the results.

Code:-

```
-- Task-04
-- write a aggregate queries to count numbers of students en
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM courses c
JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;

SELECT c.course_name
FROM course_codes c
JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;

SELECT * FROM courses;
SELECT * FROM enrollments;
```

Output:

SQL ▾			< 1 / 1 > 1 - 4 of 4	📄 ↶ ↷ 🔍
course_id	course_name	course_code		
NULL	Mathematics	MATH101		
NULL	Physics	PHYS101		
NULL	Chemistry	CHEM101		
NULL	Biology	BIO101		

SQL ▾			< 1 / 1 > 1 - 5 of 5	📄 ↶ ↷ 🔍
enrollment_id	student_id	course_id	enrollment_date	
NULL	1	1	2024-01-15	
NULL	1	2	2024-01-16	
NULL	2	1	2024-01-17	
NULL	2	3	2024-01-18	
NULL	3	4	2024-01-19	

Justification:-

The first query counts the number of students enrolled in each course by joining the Courses and Enrollments tables and grouping the results by course name. The second query filters the results to display only those courses with more than one student enrolled using the HAVING clause. The correctness of the results can be verified by comparing the query output with the data present in the Courses and Enrollments tables.

Task 5:- (Query Optimization)

Instructions:

- Use AI tools to analyze query performance.
- Rewrite inefficient sub queries using JOINS and test both queries for correctness.

Sample Code:

-- Original Query

SELECT * FROM Students

WHERE student_id IN (

SELECT student_id FROM Enrollments WHERE course_id = 101);

-- Optimized Query

SELECT s.*FROM Students s

JOIN Enrollments e ON s.student_id = e.student_id

WHERE e.course_id = 101;

Expected Output:

- Both queries return the same set of students.
- Optimized query performs better on large datasets.

Prompt:_

Analyze the performance of the queries by rewriting them using different join types such as INNER JOIN, LEFT JOIN, and RIGHT JOIN. Compare their execution plans to understand the impact of each join type on query performance, and test the results of each query.

Code:-

```
-- Task-05
--analyze the queries performance by rewriting the above qu
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);
-- Optimized Query
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;

SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output:-

SQL ▾ < 1 / 1 > 1 - 0 of 0

SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);

SQL ▾ < 1 / 1 > 1 - 0 of 0

SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;

SQL ▾ < 1 / 1 > 1 - 3 of 3

student_id	first_name	last_name	email
NULL	John	Doe	john.doe@gmail.com
NULL	Jane	Smith	jane.smith@gmail.com
NULL	Bob	Johnson	bob.johnson@gmail.com

SQL ▾ < 1 / 1 > 1 - 5 of 5

enrollment_id	student_id	course_id	enrollment_date
NULL	1	1	2024-01-15
NULL	1	2	2024-01-16
NULL	2	1	2024-01-17
NULL	2	3	2024-01-18
NULL	3	4	2024-01-19

Justifications:-

The optimized query uses an INNER JOIN to directly connect the Students and Enrollments tables based on the student_id, which is more efficient than using a subquery. The original query may require a full scan of the Students table for each student_id returned by the subquery, whereas the optimized query can leverage indexes on student_id for faster data retrieval. Additionally, the JOIN approach enables the database engine to optimize the execution plan more effectively, which can reduce execution time and improve overall performance. s